

Vector Embeddings & Semantic Search

Job Map India — complete report on every component of the semantic-search system: what it is, where it lives, where it's called from, and why each decision was made. Generated 2026-06-18.

0. Executive summary

Job matching in this app is **semantic, not keyword-based**. Every job stored in the database carries a 768-dimensional *embedding* — a numeric fingerprint of its meaning, computed from the job's title + skills + qualifications + description. When a user uploads a resume, the app extracts a skill profile, embeds that profile into the same vector space, and ranks jobs by **cosine similarity** (how close the two vectors point in 768-D space). Relevance — not keyword overlap, not recency — decides what surfaces.

The embeddings are produced by a **local model** (`Xenova/bge-base-en-v1.5`) running in-process via `Transformers.js`. There is no embedding API, no API key, no quota, and no rate limit. Similarity is computed in plain Node.js (a dot product), so no special database index or paid cluster tier is required.

Property	Value
Model	<code>Xenova/bge-base-en-v1.5</code> (BGE base, English, v1.5)
Vector size	768 dimensions, unit-normalized
Runtime	<code>@xenova/transformers</code> ^2.17.2 — local CPU, ~440 MB model, downloaded once & cached
Similarity	Cosine similarity, computed in Node (no Atlas <code>\$vectorSearch</code>)
Match threshold	<code>JOB_VECTOR_MIN_SCORE = 0.62</code> (model-specific)
Result cap	<code>JOB_VECTOR_TOP_K = 300</code> DB matches per search

Contents

1. What an embedding is, and why this app uses them
2. The pipeline at a glance — every file involved
3. The embedding client — `embeddingClient.js`
4. What text gets embedded — `jobEmbedText.js`
5. The vector fields on the Job model — `Job.js`
6. Embed-on-ingest — `jobPersistence.js`
7. Backfill script — `backfillJobEmbeddings.js`
8. DB search & ranking — `skillBasedJobSearchService.js`
9. Live-API semantic filter
10. End-to-end request flow — resume → ranked jobs
11. Storage, quota & cleanup

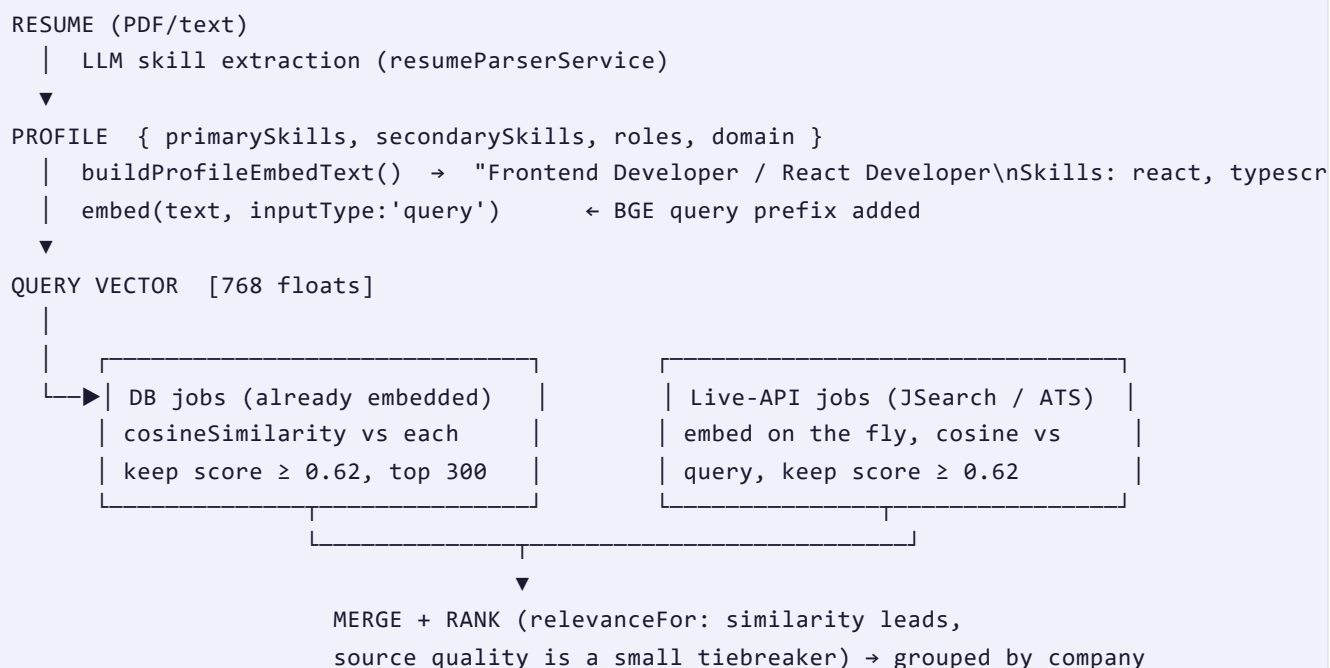
- 12. Test / diagnostic scripts
- 13. History: why local model, why app-side cosine

1. What an embedding is, and why this app uses them

An **embedding** is a list of numbers (here, 768 of them) that captures the *meaning* of a piece of text. Two texts that mean similar things produce vectors that point in nearly the same direction; unrelated texts point in different directions. The angle between two vectors — measured as **cosine similarity** (1.0 = identical direction, 0 = unrelated) — becomes a relevance score.

Why this app needs it: The old matcher was title-only keyword matching. That threw away most of the signal in a job (required skills, qualifications, description) and produced false positives — e.g. a "penetration testing" job surfacing for a React resume just because both contain the word "developer". Semantic embeddings compare the *whole meaning* of the resume against the *whole meaning* of each job, so a React profile ranks React/frontend jobs highly and pushes unrelated roles below threshold even when they share keywords.

The flow, end to end:



2. The pipeline at a glance — every file involved

Stage	File	Role
Embedding engine	<code>server/utils/embeddingClient.js</code>	Loads the local model; <code>embed()</code> turns text → vectors
What to embed	<code>server/utils/jobEmbedText.js</code>	Builds canonical job text & resume-profile text; hashing
Storage schema	<code>server/models/Job.js</code>	<code>embedding</code> , <code>embedHash</code> , <code>embeddedAt</code> fields
Embed on write	<code>server/utils/jobPersistence.js</code>	<code>embedJobIfChanged()</code> — runs at every ingest/upsert
Bulk backfill	<code>server/scripts/backfillJobEmbeddings.js</code>	Embeds all active jobs that have no vector yet
Search + rank	<code>server/services/skillBasedJobSearchService.js</code>	<code>embedProfile</code> , <code>searchDBJobs</code> , <code>cosineSimilarity</code> , <code>semanticFilterLiveJobs</code> , <code>relevanceFor</code>
HTTP entry	<code>server/routes/jobs.js</code>	POST <code>skill-search</code> → <code>searchJobsBySkills</code>
Profile source	<code>server/services/resumeParserService.js</code>	LLM extracts <code>primarySkills</code> / <code>secondarySkills</code> / <code>roles</code> / <code>domain</code>
Quota guard	<code>server/utils/dbQuota.js</code>	Detects "over quota" so embedding writes fail cleanly
Storage cap	<code>server/services/storageCleanupService.js</code>	Keeps embedded corpus under the free-tier 512 MB limit
Diagnostics	<code>server/scripts/testVectorSearch.js</code>	Runs sample profiles through <code>searchDBJobs</code>
Docs	<code>server/docs/semantic-search.md</code>	In-repo design notes (mirrors this report)

3. The embedding client `embeddingClient.js`

`server/utils/embeddingClient.js`

This is the heart of the system: the single place text becomes a vector. Everything else (ingest, backfill, search) calls `embed()`.

Constants & model choice

```
const EMBEDDING_DIMENSIONS = 768;
const MODEL_ID = process.env.LOCAL_EMBED_MODEL || "Xenova/bge-base-en-v1.5";
const QUERY_PREFIX = "Represent this sentence for searching relevant passages: ";
```

- **768 dims** — fixed output size of bge-base. Used by the model schema & backfill.
- **QUERY_PREFIX** — BGE models are *asymmetric* retrievers: the search *query* performs best when prefixed with this instruction, while *documents* (the jobs) are embedded bare. This is added only when `inputType: 'query'`. It measurably improves match quality.

Lazy singleton model loader

```
let _extractorPromise = null;
async function getExtractor() {
  if (!_extractorPromise) {
    _extractorPromise = (async () => {
      const { pipeline } = await import("@xenova/transformers"); // ESM, dynamic import
      const extractor = await pipeline("feature-extraction", MODEL_ID);
      return extractor;
    })();
  }
  return _extractorPromise;
}
```

Why a singleton: loading the ~440 MB model into memory is the expensive part. It is loaded *once* per process and reused for every embed call. The first run downloads the model to the Transformers.js cache; subsequent runs load from disk. The dynamic `import()` is required because `@xenova/transformers` is an ESM-only package in a CommonJS codebase.

The `embed()` function — the public interface

```
async function embed(input, { inputType = "document" } = {}) {
  const extractor = await getExtractor();
  const isBatch = Array.isArray(input);
  const raw = (isBatch ? input : [input]).map(t => (t || "").toString());
  const texts = inputType === "query"
    ? raw.map(t => QUERY_PREFIX + t) // query: add BGE prefix
    : raw; // document: bare

  // Mean-pool + normalize → one unit-length vector per text.
  const output = await extractor(texts, { pooling: "mean", normalize: true });

  const [n, dim] = output.dims; // Tensor shape [n, 768]
  const data = output.data;
  const vectors = [];
  for (let i = 0; i < n; i++) {
    vectors.push(Array.from(data.slice(i * dim, (i + 1) * dim)));
  }
  return isBatch ? vectors : vectors[0]; // single vector for a string, array for an array
}
```

- `inputType` is the asymmetry switch — `'document'` for jobs, `'query'` for a resume/skill profile.
- `pooling:"mean"` averages the per-token vectors into one sentence vector.
- `normalize:true` makes every vector unit length. With unit vectors, cosine similarity reduces to a plain dot product and scores land in a clean, comparable range.
- **Batch-aware:** a string in → one vector out; an array in → an array of vectors out (order preserved). The search live-filter and the backfill both pass arrays to embed a batch in a single call for speed.

Exports: `{ embed, EMBEDDING_DIMENSIONS, MODEL_ID }`.

4. What text gets embedded jobEmbedText.js

`server/utils/jobEmbedText.js`

An embedding is only as good as the text you feed it. This file defines the **canonical text** for both sides of the comparison — the job (document) and the resume profile (query) — and keeps them phrased the same way so they match well.

`buildEmbedText(job)` — the document side

```
function buildEmbedText(job = {}) {
  const parts = [];
  if (job.title) parts.push(job.title);
  if (job.requiredSkills?.length) parts.push(`Skills: ${job.requiredSkills.join(", ")}`);
  if (job.qualifications?.length) parts.push(`Qualifications: ${job.qualifications.join(", ")});
  if (job.experienceText) parts.push(job.experienceText);
  if (job.description) parts.push(job.description.slice(0, 1500));
  return parts.join("\n").trim();
}
```

Uses the *structured fields parsed at ingestion* (`requiredSkills`, `qualifications`, etc.) plus a 1500-char description snippet — exactly the signal the old title-only matcher discarded. Both ingestion and the backfill call this same function, so the embedded text never drifts between the two paths.

`buildProfileEmbedText(profile)` — the query side

```
function buildProfileEmbedText({ primarySkills = [], secondarySkills = [], roles = [], domain
  const parts = [];
  if (roles.length) parts.push(roles.join(" / "));
  const skills = [...primarySkills, ...secondarySkills];
  if (skills.length) parts.push(`Skills: ${skills.join(", ")}`);
  return parts.join("\n").trim();
}
```

Why phrased to mirror the job: the roles lead like a job-title line, and skills follow under the same `Skills:` label the job documents use. Matching the document format measurably sharpens the top results — it was A/B-tested against an older labelled phrasing (`Roles:/Also:/Domain:`) and produced a higher max similarity and a higher top-50 average.

embedTextHash(text) — change detection

```
function embedTextHash(text) {  
  return crypto.createHash("sha1").update(text || "").digest("hex");  
}
```

A stable SHA-1 of the embed text, stored as `embedHash`. Ingestion compares the new hash against the stored one; if unchanged, it skips re-embedding (saving compute). This is the mechanism that prevents redundant embedding work when a job is re-seen but its searchable content hasn't changed.

5. Vector fields on the Job model Job.js

`server/models/Job.js`

```
// Semantic search: local bge-base-en-v1.5 embedding of buildEmbedText(job)  
// (768 floats). Similarity is computed in-process via cosine, not by an Atlas  
// Vector Search index. embedHash lets ingestion skip re-embedding unchanged jobs.  
embedding: { type: [Number], default: null, select: false },  
embedHash: { type: String, default: null },  
embeddedAt: { type: Date, default: null }
```

Field	Purpose
<code>embedding</code>	The 768-float vector. <code>select: false</code> means it is <i>not</i> loaded by ordinary <code>Job.find()</code> — it's heavy (~6 KB/job), so callers must opt in with <code>.select('+embedding')</code> . Only <code>searchDBJobs</code> does.
<code>embedHash</code>	SHA-1 of the embed text — lets ingestion skip re-embedding unchanged content.
<code>embeddedAt</code>	Timestamp of last embed. Doubles as a "needs embedding?" flag: <code>embeddedAt: null</code> selects jobs the backfill still needs to process.

Comment corrected (2026-06-18): the inline comment on this field previously referenced "Voyage embedding" and an "Atlas Vector Search index" (plus a non-existent `docs/atlas-vector-search.md`) — all historical. It has been updated to describe the current reality: a local BGE embedding with cosine computed in Node. The *fields themselves* were never stale — they are the live storage for the entire semantic-search system; only the comment text was outdated.

6. Embed-on-ingest jobPersistence.js

`server/utils/jobPersistence.js`

This is where jobs *become searchable*. Every job that enters or is updated in the DB passes through here, and embedding happens automatically.

```
embedJobIfChanged(job)
```

```

async function embedJobIfChanged(job) {
  try {
    const text = buildEmbedText(job);
    if (!text) return;
    const hash = embedTextHash(text);
    if (job.embedHash === hash && job.embeddedAt) return; // unchanged → skip

    const vector = await embed(text, { inputType: "document" });
    if (!Array.isArray(vector)) return;

    await Job.updateOne(
      { _id: job._id },
      { $set: { embedding: vector, embedHash: hash, embeddedAt: new Date() } }
    );
  } catch (err) {
    console.warn(`[EMBED] Skipped job ${job?._id}:`, err?.response?.status || err?.message);
  }
}

```

- **Idempotent:** the `embedHash` check means re-seeing the same job is free.
- **Embeds as a document** (`inputType: "document"`) — no query prefix.
- **Errors are swallowed** on purpose: a missing embedding must never break ingestion. The backfill can fill any gaps later.

Where it's called

`embedJobIfChanged` is invoked at the two write paths:

- **`upsertIngestedJob(payload)`** — the main ingestion entry. After the job is upserted (and its description parsed into `requiredSkills` / `qualifications` /etc.), it `await s embedJobIfChanged`. It is awaited so a job is never returned as searchable before it can actually be matched.
- **`touchExistingJob(job, updates)`** — when a re-seen job gets a newly-filled description (which changes the embed text), it re-embeds.

Every ingestion source (Naukri, Adzuna, ATS scrapers, universal scrapers) ultimately routes job writes through `upsertIngestedJob`, so they all inherit embedding for free without referencing the embedding system directly.

7. Backfill script `backfillJobEmbeddings.js`

`server/scripts/backfillJobEmbeddings.js` • run: `node server/scripts/backfillJobEmbeddings.js`

One-off / re-runnable job that embeds every active job which doesn't yet have a vector. Used to seed the corpus and to fill gaps after model changes or ingestion hiccups.

```

const filter = { isActive: true, embeddedAt: null }; // every job missing a vector, no time

while (true) {
  const jobs = await Job.find(filter)
    .select("title requiredSkills qualifications experienceText description")
    .limit(BATCH_SIZE).lean();
  if (jobs.length === 0) break;

  const prepared = jobs.map(job => ({ job, text: buildEmbedText(job) })).filter(p => p.text);

  // Jobs with no embeddable text: stamp embeddedAt so the loop terminates.
  const emptyIds = jobs.filter(j => !buildEmbedText(j)).map(j => j._id);
  if (emptyIds.length) await Job.updateMany({ _id: { $in: emptyIds } }, { $set: { embeddedAt:

  if (prepared.length) {
    const vectors = await embed(prepared.map(p => p.text), { inputType: "document" }); // bat
    const ops = prepared.map((p, i) => ({ updateOne: {
      filter: { _id: p.job._id },
      update: { $set: { embedding: vectors[i], embedHash: embedTextHash(p.text), embeddedAt: n
    } }));
    await Job.bulkWrite(ops); // with ECONNRESET retry (up to 5x)
  }
}

```

- **No time window** — embeds the entire active corpus, not just recent jobs.
- **Batched** by `BACKFILL_BATCH_SIZE` (default 32) — controls embeds + DB writes per round-trip.
- **Safe to re-run** — only touches jobs where `embeddedAt` is null.
- **Empty-text jobs** get `embeddedAt` stamped (but no vector) so the loop terminates and they aren't retried forever.
- **Write retries** — Atlas connections can drop mid-run (`ECONNRESET`); writes retry up to 5× since the vectors are already computed and re-writing is idempotent.

8. DB search & ranking skillBasedJobSearchService.js

`server/services/skillBasedJobSearchService.js`

This is where embeddings are *used* — turned into ranked results.

Tunables

```

const MIN_VECTOR_SCORE = Number(process.env.JOB_VECTOR_MIN_SCORE || 0.62);
const VECTOR_TOP_K      = Number(process.env.JOB_VECTOR_TOP_K      || 300);

```

Why 0.62: bge-base has a high similarity *baseline* — unrelated professional text (sales/HR/procurement roles) scored ~0.50–0.60 against a React resume in testing. Real signal starts ~0.62+. The threshold cuts that noise while keeping recall for thin/niche domains. **It is model-specific** — if the embedding model changes, this must be re-measured.

cosineSimilarity(a, b) — the math, in Node

```
function cosineSimilarity(a, b) {
  if (!Array.isArray(a) || !Array.isArray(b) || a.length !== b.length) return -1;
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) { dot += a[i]*b[i]; na += a[i]*a[i]; nb += b[i]*b[i]; }
  if (na === 0 || nb === 0) return -1;
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
}
```

Why in-process, not Atlas \$vectorSearch : Atlas Vector Search needs a dedicated M10+ cluster (not available on free tiers). Computing cosine in Node works on *any* cluster tier including the free M0. At the current corpus size (~18k jobs) the per-search cost of loading and scoring vectors is small. The docs flag this as the one place to revisit (move to a vector index / pgvector) if the corpus grows into the tens of thousands.

embedProfile(...) — resume → query vector

```
async function embedProfile(primarySkills, secondarySkills, suggestedRoles, domain) {
  const profileText = buildProfileEmbedText({ primarySkills, secondarySkills, roles: suggestedRoles });
  if (!profileText) return null;
  return await embed(profileText, { inputType: "query" }); // ← query prefix applied
}
```

Embedded **once** per search and reused for both the DB scan and the live-API filter, so the profile is never embedded twice.

searchDBJobs(...) — the core retrieval

```
const candidates = await Job.find({ isActive: true, embedding: { $ne: null } })
  .select('+embedding title location applyLink source description firstSeenAt lastSeenAt ' +
    'postedDate experienceLevel yearsMin yearsMax experienceText requiredSkills ' +
    'qualifications responsibilityBullets company')
  .populate('company', 'name logo domain lat lng atsProvider hiringVelocity careersProvider')
  .lean();

const jobs = candidates
  .map(job => ({ job, _score: cosineSimilarity(queryVector, job.embedding) }))
  .filter(x => x._score >= MIN_VECTOR_SCORE) // threshold
  .sort((a, b) => b._score - a._score) // best first
  .slice(0, VECTOR_TOP_K) // cap at 300
  .map(x => ({ ...x.job, _score: x._score }));
```

Steps: load *all* embedded active jobs (no time window — relevance, not age, decides), score each by cosine against the query vector, keep those ≥ 0.62 , sort by score, keep the top 300. Then two post-filters before returning:

- **Hard title exclusion:** jobs whose title matches `IRRELEVANT_TITLES` (officer, director, VP, sales, HR, recruiter, ...) are dropped *regardless* of similarity — the one keyword rule kept, to guarantee leadership/non-tech roles never surface.

- **Per-source freshness:** each source has a max age (Naukri 5 days, JSearch 10, Adzuna 14, direct ATS 45, default 21); stale jobs are skipped.

Survivors are shaped into result objects carrying `_vectorScore` (the cosine score) plus company coords, ATS provider, experience metadata, etc.

`relevanceFor(job, ...)` — final ranking

```
function relevanceFor(job, primarySkills, suggestedRoles, domain) {
  const sourceBonus = ((SOURCE_QUALITY[job.source] || 60) - 60) / 35 * 12; // 0-12
  if (typeof job._vectorScore === 'number') {
    return job._vectorScore * 100 + sourceBonus; // semantic leads (0-100), source is a
  }
  // No vector (live-API fallback): keyword score → comparable 0-100 band + same bonus
  const kw = scoreJob(job.title, job.location, job.source, primarySkills, suggestedRoles, domain);
  return Math.min((kw - 60) / 110 * 100, 100) * 0.85 + sourceBonus;
}
```

Why semantic-led: when a job has a real vector score, similarity (scaled 0–100) is the primary axis and source quality contributes only ~0–12 as a tiebreaker. So a clearly better semantic match always outranks a merely better-sourced one — but among similar matches, the more trustworthy source (direct ATS > aggregator) wins. Live-API jobs without a vector fall back to the keyword score, mapped into the same band so the two kinds interleave sensibly instead of clustering all-DB-then-all-API.

9. Live-API semantic filter

JSearch and the ATS boards (Greenhouse/Lever/Ashby/SmartRecruiters) are fetched in real time and aren't in the embedded DB. They're first keyword pre-filtered (`isTitleRelevant` , cheap), then **semantically filtered** against the same 0.62 threshold so the same relevance bar governs live and DB results alike.

```
async function semanticFilterLiveJobs(jobs, queryVector) {
  const texts = jobs.map(buildLiveJobEmbedText); // title + skills + desc snip
  const vectors = await embed(texts, { inputType: 'document' }); // one batched call
  const kept = [];
  for (let i = 0; i < jobs.length; i++) {
    const score = cosineSimilarity(queryVector, vectors[i]);
    if (score >= MIN_VECTOR_SCORE) kept.push({ ...jobs[i], _vectorScore: score });
  }
  return kept;
}
```

- `buildLiveJobEmbedText` mirrors the document side (title leads, skills follow) so live jobs are comparable to the resume query vector.
- **This is what stops keyword-only junk** — e.g. a pentest job that passed the title filter for a React resume gets a low cosine score and is dropped.
- **Fail-open:** if embedding throws, the keyword-filtered jobs are returned unchanged — better to show them than to break the search.

- DB jobs already carry `_vectorScore` from `searchDBJobs`, so they are *not* re-embedded here.

10. End-to-end request flow — resume → ranked jobs

1. **Resume parsed** (`resumeParserService.js`) — an LLM extracts `{ domain, primarySkills, secondarySkills, roles }`. `primarySkills` = domain-defining tools (max 15), `secondarySkills` = general languages/tools (max 10). This profile is the raw material for the query embedding.
2. **HTTP entry** (`routes/jobs.js`) — the skill-search endpoint reads `{ skills, roles, primarySkills, secondarySkills, domain }` from the body and calls `searchJobsBySkills(...)`.
3. `searchJobsBySkills` orchestrates:
 - a. Cache check — identical recent searches return cached results (skips embedding entirely).
 - b. `embedProfile(...)` — embed the resume profile *once* → query vector.
 - c. `searchDBJobs(...)` — cosine scan over embedded DB jobs.
 - d. Live APIs (JSearch + ATS) fetched in parallel, keyword pre-filtered, then `semanticFilterLiveJobs(...)`.
 - e. Merge DB + surviving live jobs → `persistAndGroupJobs` groups by company & ranks via `relevanceFor`.
4. **Response** — companies (with their best-ranked roles) returned to the client and cached.

11. Storage, quota & cleanup

Each embedding is 768 floats $\approx$ ~6 KB/job. On the free MongoDB Atlas tier (512 MB), vectors + descriptions add up, so three safeguards keep storage bounded:

- `dbQuota.js` — `isQuotaError()` recognizes Atlas's "over space quota — writes are blocked" message so ingestion (and embedding writes) stop cleanly instead of spamming thousands of failures. `DbQuotaFullError` is thrown to abort a run at the top level.
- `storageCleanupService.js` — runs daily: deletes inactive jobs past a grace period (`STORAGE_INACTIVE_GRACE_DAYS`, default 3) and hard-caps total job count (`STORAGE_MAX_JOBS`, default 40k) by trimming the oldest.
- **Manual** — deleting inactive/non-embedded jobs frees space; writes unblock once back under quota.

Note: embedding is gated on staying under quota — when the DB is full, `embedJobIfChanged`'s `Job.updateOne` fails, but it swallows the error so ingestion continues; the backfill re-fills once space is freed.

12. Test / diagnostic scripts

`server/scripts/testVectorSearch.js` • run: `node server/scripts/testVectorSearch.js`

Runs three hardcoded sample profiles (Frontend React/TS, Data Science Python/ML, Backend Java/Spring) through the real `searchDBJobs` and prints the top 10 matches with their cosine scores. Useful for sanity-checking the corpus, the threshold, and relevance after any change to the model or embed text.

```
const results = await searchDBJobs(p.primarySkills, p.secondarySkills, p.roles, p.domain);
results.slice(0, 10).forEach((r, i) => {
  const score = r._vectorScore?.toFixed(3) ?? "-";
  console.log(`  ${i+1}. [${score}] ${r.job_title} @ ${r.employer_name} (${r.source})`);
});
```

It also prints `countDocuments({ embeddedAt: { $ne: null } })` so you can see how many jobs are actually searchable.

13. History — why local model, why app-side cosine

Decision	Rationale
Semantic over keyword	Title-only keyword matching discarded skills/qualifications/description and produced false positives on shared words. Embeddings compare whole meaning.
Local model (BGE) over hosted	Hosted free tiers (Voyage, Gemini) were tried first but their daily caps couldn't keep up with the ingestion volume. The local model has no API, key, quota, or rate limit — every job in the corpus can be embedded.
App-side cosine over Atlas <code>\$vectorSearch</code>	Atlas Vector Search requires a paid M10+ cluster. In-process cosine runs on the free M0 tier; cheap enough at ~18k jobs.
Asymmetric query/document	BGE retrieves best with a query-instruction prefix on the search side and bare documents — handled by <code>inputType</code> .
0.62 threshold	Empirically tuned to bge-base's high similarity baseline; re-measure if the model changes.

Current state in git: the embedding files (`embeddingClient.js` , `jobEmbedText.js` , the backfill & test scripts, `dbQuota.js` , `docs/semantic-search.md`) are *untracked / uncommitted* on the `main` branch — this is a newly added subsystem. The `Job.js` model and `skillBasedJobSearchService.js` are tracked and modified. The dependency `@xenova/transformers ^2.17.2` is in `server/package.json`.

Report covers: `embeddingClient.js` · `jobEmbedText.js` · `Job.js` · `jobPersistence.js` · `backfillJobEmbeddings.js` · `skillBasedJobSearchService.js` · `routes/jobs.js` · `resumeParserService.js` · `dbQuota.js` · `storageCleanupService.js` · `testVectorSearch.js` · `docs/semantic-search.md`.